

PROMPT MAESTRO PARA CODEX

Proyecto: FollowRead

Quiero que actúes como arquitecto de software, desarrollador full-stack senior, diseñador UX/UI, especialista en accesibilidad, especialista en aplicaciones móviles híbridas, especialista en Amazon Web Services y mentor de desarrolladores junior.

Debes diseñar y desarrollar un proyecto completo llamado **FollowRead**, siguiendo estrictamente las reglas, arquitectura, fases, tareas, documentación y criterios definidos en este documento.

No debes intentar construir todo el proyecto en una sola ejecución. El proyecto debe desarrollarse por fases controladas, y cada fase debe contener actividades y tareas claramente identificadas, documentadas y administradas mediante estados.

Antes de modificar código, debes revisar el estado actual del proyecto, identificar la fase activa y continuar desde la primera tarea pendiente o bloqueada que corresponda.

1. NOMBRE DEL PROYECTO

FollowRead

2. DESCRIPCIÓN GENERAL

FollowRead será una plataforma de lectura interactiva que permitirá leer cuentos infantiles, artículos, documentos y libros para adultos.

La aplicación reproducirá el contenido mediante audio y mostrará visualmente el avance de la lectura.

Mientras el audio se reproduce:

- la palabra pronunciada debe resaltarse;
- una mano animada con un dedo debe señalar la palabra activa;
- el texto debe desplazarse automáticamente cuando sea necesario;
- el usuario debe poder pausar, continuar, retroceder y repetir;
- el progreso de lectura debe guardarse;
- el usuario debe poder cambiar la velocidad de reproducción;
- el contenido puede estar disponible en inglés, español o ambos idiomas.

FollowRead también tendrá un modo especial para aprender inglés.

En este modo:

- el texto principal estará en inglés;

- la traducción al español podrá mostrarse como apoyo;
 - cada palabra podrá tocarse para escucharla;
 - se podrá consultar su traducción;
 - se podrá repetir una oración;
 - se podrá reducir la velocidad;
 - el usuario podrá guardar palabras en su vocabulario personal.
-

3. COMPONENTES PRINCIPALES

El proyecto debe estar dividido en los siguientes componentes.

3.1. FollowRead Admin

Aplicación web privada para administrar contenido.

Debe permitir:

- iniciar sesión como administrador;
- crear cuentos;
- crear artículos;
- crear libros;
- crear capítulos;
- crear páginas, secciones y párrafos;
- crear contenido en inglés;
- crear contenido en español;
- asociar traducciones;
- seleccionar la audiencia;
- seleccionar el nivel de lectura;
- seleccionar las voces de Amazon Polly;
- generar audio;
- generar marcas de tiempo;
- revisar la sincronización;
- agregar portadas;
- agregar ilustraciones;
- guardar borradores;
- previsualizar contenido;
- publicar;
- despublicar;
- archivar;
- crear nuevas versiones del contenido;
- consultar errores durante la generación del audio;
- volver a procesar contenido con errores.

El administrador existirá solamente en la versión web.

No debe estar incluido dentro de la aplicación móvil.

3.2. FollowRead Reader

Aplicación utilizada por los lectores.

Debe funcionar como:

- aplicación web;
- Progressive Web App;
- aplicación Android mediante Capacitor;
- aplicación iOS mediante Capacitor.

Debe incluir:

- biblioteca;
 - categorías;
 - búsqueda;
 - filtros;
 - detalles del contenido;
 - lector interactivo;
 - reproducción de audio;
 - resaltado palabra por palabra;
 - mano animada;
 - guardado del progreso;
 - favoritos;
 - historial;
 - configuración;
 - modo infantil;
 - modo adulto;
 - modo español;
 - modo inglés;
 - modo aprender inglés;
 - descarga de contenido para uso offline;
 - sincronización con el backend;
 - manejo de contenido incluido dentro del build;
 - manejo de contenido descargado posteriormente.
-

3.3. FollowRead API

Backend central del sistema.

Debe encargarse de:

- autenticación;
- autorización;
- usuarios;
- administradores;
- contenido;
- idiomas;
- traducciones;

- capítulos;
 - párrafos;
 - versiones;
 - estados de publicación;
 - integración con Amazon Polly;
 - generación de audio;
 - generación de Speech Marks;
 - almacenamiento de archivos;
 - catálogo;
 - sincronización de contenido;
 - descargas;
 - progreso de usuarios;
 - favoritos;
 - vocabulario;
 - registros de errores;
 - auditoría;
 - documentación de API.
-

4. TECNOLOGÍAS PRINCIPALES

Frontend

- React
- TypeScript
- Vite
- HTML5
- CSS3
- Capacitor
- React Router
- TanStack Query
- Zustand
- React Hook Form
- Zod
- Framer Motion
- Vitest
- React Testing Library
- Playwright

Puedes recomendar librerías adicionales, pero debes justificar claramente cada dependencia antes de agregarla.

Evita instalar librerías innecesarias.

Backend

Utiliza:

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- PostgreSQL
- Pytest
- Docker

Para tareas largas, como generación de audio, prepara la arquitectura para utilizar:

- Redis;
- Celery o una alternativa equivalente.

La primera versión puede implementar una solución simple, pero la arquitectura debe permitir agregar procesamiento asíncrono posteriormente.

Amazon Web Services

Utiliza:

- Amazon Polly para Text-to-Speech;
- Amazon Polly Speech Marks para sincronización;
- Amazon S3 para almacenar audio, imágenes y paquetes de contenido.

Nunca debes exponer credenciales de AWS en el frontend, repositorio o aplicación móvil.

Las credenciales deben manejarse exclusivamente desde el backend mediante variables de entorno o roles seguros.

5. ARQUITECTURA DEL REPOSITORIO

Utiliza un monorepo organizado de la siguiente manera:

```
followread/  
|  
├─ apps/  
|   ├─ admin-web/  
|   ├─ reader/  
|   └─ api/  
|  
├─ packages/  
|   └─ shared-types/
```

```

|   |   | shared-ui/
|   |   | content-models/
|   |   | reader-engine/
|   |   | validation/
|   |   | └─ configuration/
|   |
|   └─ infrastructure/
|       |   | docker/
|       |   | aws/
|       |   | database/
|       |   | └─ deployment/
|       |
|       └─ docs/
|           |   | requirements/
|           |   | architecture/
|           |   | development/
|           |   | troubleshooting/
|           |   | api/
|           |   | user-guides/
|           |   | project-management/
|           |   | ux-ui/
|           |   | testing/
|           |   | └─ deployment/
|           |
|           └─ scripts/
|               |
|               └─ .github/
|                   |   | workflows/
|                   |   | ISSUE_TEMPLATE/
|                   |   | └─ PULL_REQUEST_TEMPLATE.md
|                   |
|                   └─ docker-compose.yml
|                       |
|                       └─ README.md
|                           |
|                           └─ CONTRIBUTING.md
|                               |
|                               └─ CHANGELOG.md
|                                   |
|                                   └─ LICENSE
|                                       |
|                                       └─ SECURITY.md

```

Puedes mejorar esta estructura cuando sea necesario, pero no debes romper la separación entre Admin, Reader y API.

6. PRINCIPIO DE ACTUALIZACIÓN DE CONTENIDO

FollowRead debe manejar contenido dinámico.

Agregar o corregir contenido no debe requerir un nuevo build de la aplicación móvil.

La arquitectura debe seguir esta regla:

Nuevo cuento o artículo:
No requiere nuevo build.

Corrección de contenido:
No requiere nuevo build.

Nuevo audio:
No requiere nuevo build.

Nueva portada o ilustración:
No requiere nuevo build.

Nueva funcionalidad:
Sí puede requerir nuevo build.

Cambios de permisos móviles:
Sí requieren nuevo build.

Actualización de Capacitor:
Sí requiere nuevo build.

La aplicación debe incluir contenido inicial dentro del build, pero también debe consultar el backend para detectar nuevo contenido o nuevas versiones.

7. SISTEMA DE VERSIONES DE CONTENIDO

Cada elemento publicado debe incluir:

- identificador único;
- versión;
- fecha de creación;
- fecha de actualización;
- estado;
- idioma;
- checksum;
- URL de descarga;
- requisitos mínimos de la aplicación;
- información de compatibilidad.

Ejemplo conceptual:

```
{  
  "id": "story-001",  
  "version": 3,  
  "contentType": "story",
```

```
"status": "published",  
"languages": ["en", "es"],  
"minimumAppVersion": "1.0.0",  
"updatedAt": "2026-07-24T12:00:00Z",  
"checksum": "sha256-value",  
"packageUrl": "/content/story-001/version/3/package.json"  
}
```

La aplicación Reader debe:

1. leer el catálogo local;
2. consultar el catálogo remoto;
3. comparar versiones;
4. descargar solo contenido nuevo o modificado;
5. verificar integridad;
6. almacenar el contenido;
7. permitir uso offline;
8. conservar el progreso del usuario;
9. manejar errores de actualización sin bloquear la aplicación.

8. TIPOS DE CONTENIDO

El sistema debe soportar:

- story;
- article;
- book;
- lesson.

Las audiencias iniciales serán:

- children;
- teenager;
- adult;
- all.

Los niveles de lectura serán:

- beginner;
- elementary;
- intermediate;
- upper-intermediate;
- advanced.

Los idiomas iniciales serán:

- English;
- Spanish.

La arquitectura debe permitir agregar otros idiomas en el futuro.

9. MODOS DE LECTURA

9.1. Modo español

- texto en español;
- narración en español;
- resaltado en español;
- controles de reproducción;
- progreso.

9.2. Modo inglés

- texto en inglés;
- narración en inglés;
- resaltado en inglés;
- controles de reproducción;
- progreso.

9.3. Modo aprender inglés

- texto principal en inglés;
- traducción opcional al español;
- audio en inglés;
- repetición de palabras;
- repetición de oraciones;
- velocidad reducida;
- vocabulario;
- traducción contextual;
- marcación de palabras desconocidas.

9.4. Modo infantil

- tipografía grande;
- controles grandes;
- interfaz simple;
- mano animada visible;
- ilustraciones;
- navegación segura;
- pocas opciones en pantalla;
- ausencia de elementos distractores;
- transiciones suaves;
- lectura automática opcional.

9.5. Modo adulto

- diseño sobrio;

- mano opcional;
 - resaltado configurable;
 - tipografía ajustable;
 - modo oscuro;
 - notas;
 - marcadores;
 - lectura por palabra, oración o párrafo.
-

10. MOTOR DE LECTURA

Debes crear un motor reutilizable llamado:

reader-engine

Debe encargarse de:

- reproducir audio;
- identificar el tiempo actual;
- encontrar la palabra activa;
- resaltar la palabra;
- mover el puntero;
- desplazar el contenido;
- manejar pausa;
- reanudar;
- avanzar;
- retroceder;
- repetir palabra;
- repetir oración;
- cambiar velocidad;
- sincronizar progreso;
- recuperar la posición anterior;
- manejar errores de audio;
- cambiar entre capítulos.

Debe estar desacoplado de la interfaz gráfica.

Debe poder probarse de forma independiente.

11. MANO ANIMADA

La mano debe implementarse inicialmente como SVG.

Debe:

- señalar la palabra activa;
- moverse suavemente;

- permanecer debajo de la palabra;
- adaptarse a diferentes tamaños de pantalla;
- manejar saltos entre líneas;
- ocultarse cuando el usuario lo desactive;
- no tapar el texto;
- respetar preferencias de reducción de movimiento;
- funcionar en web, Android e iOS.

Utiliza `getBoundingClientRect()` o una solución equivalente para localizar la palabra activa.

Usa Framer Motion o animaciones CSS para el desplazamiento.

Debes incluir documentación detallada explicando:

- cómo se calcula la posición;
- cómo se identifica la palabra activa;
- cómo se mueve la mano;
- cómo se adapta al scroll;
- cómo se corrigen problemas de desalineación.

12. AMAZON POLLY

El backend debe implementar una capa de abstracción para Amazon Polly.

No conectes directamente los controladores HTTP con AWS.

Utiliza una estructura similar a:

```
PollyService
SpeechGenerationService
SpeechMarksParser
AudioStorageService
ContentProcessingService
```

El flujo será:

1. el administrador crea el texto;
2. selecciona idioma y voz;
3. solicita generar audio;
4. el backend valida el contenido;
5. el backend divide el texto en unidades apropiadas;
6. se genera el audio;
7. se generan Speech Marks;
8. se almacenan audio y marcas;
9. se procesa la relación entre texto y marcas;
10. se ejecutan validaciones;
11. el contenido pasa a revisión;

12. el administrador previsualiza;
13. el administrador publica.

Debe existir manejo de:

- límites de texto;
- caracteres incompatibles;
- errores de AWS;
- reintentos;
- archivos incompletos;
- inconsistencias entre palabras y Speech Marks;
- generación parcial;
- cancelación;
- reprocesamiento.

13. ESTADOS DEL CONTENIDO

El contenido debe manejar los siguientes estados:

```
draft
ready_for_processing
processing
processing_failed
ready_for_review
review_rejected
approved
published
unpublished
archived
```

Cada cambio de estado debe:

- validarse;
- registrarse;
- incluir fecha;
- incluir usuario responsable;
- incluir comentario opcional;
- impedir transiciones inválidas.

Crea una máquina de estados o una estrategia equivalente.

Documenta todas las transiciones permitidas.

14. ADMINISTRACIÓN DE FASES Y TAREAS

Este proyecto debe incluir un sistema documental para que Codex sepa:

- qué fase está activa;
- qué tareas están pendientes;
- qué tareas están en progreso;
- qué tareas están terminadas;
- cuáles están bloqueadas;
- qué decisiones se tomaron;
- qué problemas siguen abiertos;
- desde dónde debe continuar.

Crea los siguientes archivos:

```
docs/project-management/PROJECT_STATUS.md
docs/project-management/PHASES.md
docs/project-management/TASKS.md
docs/project-management/DECISIONS.md
docs/project-management/RISKS.md
docs/project-management/KNOWN_ISSUES.md
docs/project-management/NEXT_STEPS.md
docs/project-management/SESSION_LOG.md
```

15. ESTADOS DE LAS TAREAS

Cada tarea debe usar uno de estos estados:

```
NOT_STARTED
IN_PROGRESS
BLOCKED
READY_FOR_REVIEW
COMPLETED
CANCELLED
DEFERRED
```

Cada tarea debe contener:

```
ID:
Título:
Fase:
Descripción:
Objetivo:
Estado:
Prioridad:
```

Dependencias:
Archivos relacionados:
Criterios de aceptación:
Pruebas requeridas:
Documentación requerida:
Problemas encontrados:
Decisiones tomadas:
Fecha de inicio:
Fecha de finalización:
Siguiendo acción:

Ejemplo:

```
## FR-PH02-TASK-004

**Título:** Crear el modelo de contenido bilingüe
**Fase:** 2
**Estado:** IN_PROGRESS
**Prioridad:** High

### Descripción
Crear las entidades necesarias para almacenar contenido en inglés y español.

### Dependencias
- FR-PH02-TASK-001
- FR-PH02-TASK-002

### Archivos relacionados
- apps/api/app/models/content.py
- packages/shared-types/src/content.ts

### Criterios de aceptación
- El contenido puede tener una versión en inglés.
- El contenido puede tener una versión en español.
- Los idiomas pueden consultarse independientemente.
- Existen validaciones.
- Existen pruebas unitarias.
- Existe documentación.

### Siguiendo acción
Implementar las migraciones de base de datos.
```

16. REGLAS PARA CODEX AL INICIAR CADA SESIÓN

Antes de escribir o modificar código debes:

1. leer `PROJECT_STATUS.md` ;
2. leer `PHASES.md` ;
3. leer `TASKS.md` ;
4. leer `NEXT_STEPS.md` ;
5. leer las últimas entradas de `SESSION_LOG.md` ;
6. revisar `KNOWN_ISSUES.md` ;
7. revisar `DECISIONS.md` ;
8. identificar la fase activa;
9. identificar la primera tarea que se pueda ejecutar;
10. verificar dependencias;
11. explicar brevemente qué vas a hacer;
12. ejecutar solamente las tareas apropiadas.

No debes marcar una tarea como completada si:

- el código no funciona;
- faltan pruebas;
- las pruebas fallan;
- falta documentación;
- no se cumplieron los criterios de aceptación;
- hay errores conocidos no documentados;
- faltan migraciones;
- faltan instrucciones de ejecución.

Al finalizar cada sesión debes:

1. actualizar el estado de las tareas;
2. actualizar `PROJECT_STATUS.md` ;
3. actualizar `NEXT_STEPS.md` ;
4. registrar decisiones;
5. registrar errores conocidos;
6. registrar riesgos;
7. actualizar `SESSION_LOG.md` ;
8. documentar comandos ejecutados;
9. documentar pruebas ejecutadas;
10. indicar exactamente desde dónde debe continuar la siguiente sesión.

17. FASES DEL PROYECTO

FASE 0 — Descubrimiento, definición y planificación

Objetivo:

Definir completamente el producto antes de iniciar el desarrollo.

Actividades:

- definir visión;
- definir problema;
- definir usuarios;
- definir alcance;
- definir fuera de alcance;
- definir funcionalidades;
- definir requerimientos funcionales;
- definir requerimientos no funcionales;
- definir casos de uso;
- definir historias de usuario;
- definir criterios de aceptación;
- definir riesgos;
- definir dependencias;
- definir arquitectura inicial;
- definir experiencia de usuario;
- definir accesibilidad;
- definir estrategia de pruebas;
- definir estrategia de seguridad;
- definir estrategia de despliegue;
- definir plan de fases.

Entregables:

```
docs/requirements/PRODUCT_VISION.md
docs/requirements/PROJECT_SCOPE.md
docs/requirements/FUNCTIONAL_REQUIREMENTS.md
docs/requirements/NON_FUNCTIONAL_REQUIREMENTS.md
docs/requirements/USER_STORIES.md
docs/requirements/USE_CASES.md
docs/requirements/ACCEPTANCE_CRITERIA.md
docs/architecture/SYSTEM_CONTEXT.md
docs/ux-ui/UX_STRATEGY.md
docs/testing/TEST_STRATEGY.md
docs/project-management/PHASES.md
docs/project-management/TASKS.md
```

FASE 1 — Diseño UX/UI y sistema visual

Objetivo:

Diseñar la experiencia antes de implementar pantallas definitivas.

Actividades:

- crear perfiles de usuario;
- crear journeys;
- crear mapas de navegación;
- crear wireframes;
- crear diseño responsive;
- crear sistema visual;
- definir tipografías;
- definir espaciado;
- definir iconografía;
- definir componentes;
- definir estados vacíos;
- definir estados de carga;
- definir estados de error;
- definir experiencia infantil;
- definir experiencia adulta;
- definir experiencia de aprendizaje;
- definir accesibilidad;
- definir reducción de movimiento;
- definir contraste;
- definir navegación por teclado;
- definir tamaños táctiles.

Pantallas mínimas del Reader:

- inicio;
- biblioteca;
- categorías;
- búsqueda;
- detalle del contenido;
- lector;
- descargas;
- favoritos;
- historial;
- vocabulario;
- configuración;
- perfil.

Pantallas mínimas del Admin:

- inicio de sesión;
- dashboard;
- lista de contenidos;
- crear contenido;
- editar contenido;
- capítulos;
- traducciones;
- configuración de voz;
- procesamiento de audio;
- revisión;
- previsualización;

- publicación;
- historial;
- errores.

Entregables:

```
docs/ux-ui/DESIGN_SYSTEM.md
docs/ux-ui/USER_FLOWS.md
docs/ux-ui/WIREFRAMES.md
docs/ux-ui/ACCESSIBILITY.md
docs/ux-ui/RESPONSIVE_DESIGN.md
docs/ux-ui/CHILDREN_MODE.md
docs/ux-ui/ADULT_MODE.md
docs/ux-ui/LEARNING_MODE.md
```

FASE 2 — Preparación del monorepo y entorno de desarrollo

Objetivo:

Crear la estructura base y herramientas del proyecto.

Actividades:

- crear monorepo;
- configurar frontend;
- configurar backend;
- configurar TypeScript;
- configurar linting;
- configurar formatting;
- configurar pruebas;
- configurar variables de entorno;
- configurar Docker;
- configurar PostgreSQL;
- configurar migraciones;
- configurar scripts;
- configurar Git hooks;
- configurar CI;
- crear documentación de instalación.

Entregables:

- proyectos compilando;
- pruebas base funcionando;
- Docker funcionando;
- documentación de configuración;
- archivo `.env.example`;
- scripts de desarrollo;
- pipeline de validación.

FASE 3 — Modelado de datos y API base

Objetivo:

Crear la base funcional del backend.

Actividades:

- diseñar modelos;
- implementar base de datos;
- implementar migraciones;
- implementar repositorios;
- implementar servicios;
- implementar validaciones;
- implementar respuestas estándar;
- implementar manejo de errores;
- implementar logging;
- implementar health checks;
- implementar OpenAPI;
- implementar pruebas.

Entidades iniciales:

- User;
- Administrator;
- Role;
- Permission;
- ReadingContent;
- ContentTranslation;
- Chapter;
- Paragraph;
- AudioAsset;
- SpeechMark;
- Illustration;
- Category;
- ReadingLevel;
- ContentVersion;
- Publication;
- ProcessingJob;
- AuditLog;
- ReadingProgress;
- Favorite;
- VocabularyWord;
- DownloadRecord.

FASE 4 — Autenticación y autorización

Objetivo:

Proteger Admin y preparar usuarios del Reader.

Actividades:

- login;
- logout;
- tokens;
- refresh tokens;
- contraseñas seguras;
- recuperación;
- roles;
- permisos;
- protección de rutas;
- auditoría;
- manejo seguro de sesiones;
- pruebas de seguridad;
- documentación.

Roles iniciales:

- super_admin;
- content_admin;
- reviewer;
- reader.

FASE 5 — FollowRead Admin

Objetivo:

Construir el administrador web.

Actividades:

- dashboard;
- CRUD de contenido;
- editor de capítulos;
- editor de párrafos;
- editor bilingüe;
- selección de voces;
- subida de imágenes;
- validaciones;
- borradores;
- estados;
- filtros;
- búsqueda;
- previsualización;
- historial;
- publicación;
- experiencia responsive;

- manejo de errores;
- pruebas.

El editor debe minimizar el riesgo de pérdida de información.

Debe implementar:

- autoguardado;
- confirmación antes de abandonar;
- indicadores de guardado;
- validaciones claras;
- recuperación de borrador.

FASE 6 — Integración con Amazon Polly

Objetivo:

Generar audio y marcas de sincronización.

Actividades:

- configurar cliente AWS;
- implementar PollyService;
- implementar Speech Marks;
- dividir texto;
- generar audio;
- almacenar audio;
- almacenar marcas;
- mapear texto;
- validar sincronización;
- manejar errores;
- permitir reintentos;
- mostrar progreso;
- registrar costos estimados;
- crear pruebas con mocks;
- documentar configuración.

Nunca uses servicios reales de AWS durante las pruebas automatizadas.

Utiliza mocks, fixtures o adaptadores falsos.

FASE 7 — Motor de lectura

Objetivo:

Implementar el núcleo técnico de FollowRead.

Actividades:

- cargar contenido;
- cargar audio;
- interpretar Speech Marks;
- calcular palabra activa;
- resaltar palabras;
- mover la mano;
- auto-scroll;
- pausar;
- continuar;
- repetir;
- cambiar velocidad;
- cambiar capítulo;
- recuperar progreso;
- manejar orientación;
- manejar cambio de tamaño;
- manejar interrupciones;
- manejar pérdida de audio;
- pruebas unitarias;
- pruebas de integración;
- documentación profunda.

FASE 8 — FollowRead Reader Web

Objetivo:

Construir la aplicación de lectura web.

Actividades:

- biblioteca;
- categorías;
- búsqueda;
- detalles;
- lector;
- favoritos;
- progreso;
- historial;
- configuración;
- vocabulario;
- modos de lectura;
- diseño responsive;
- PWA;
- accesibilidad;
- pruebas E2E.

FASE 9 — Sincronización y modo offline

Objetivo:

Permitir descargar y utilizar contenido sin conexión.

Actividades:

- catálogo remoto;
 - catálogo local;
 - comparación de versiones;
 - descarga;
 - almacenamiento;
 - checksum;
 - eliminación;
 - actualización;
 - recuperación ante fallos;
 - almacenamiento limitado;
 - progreso offline;
 - sincronización posterior;
 - indicador de conexión;
 - pruebas.
-

FASE 10 — Capacitor, Android e iOS

Objetivo:

Convertir el Reader en aplicación móvil.

Actividades:

- configurar Capacitor;
- crear proyecto Android;
- crear proyecto iOS;
- configurar iconos;
- configurar splash screen;
- configurar almacenamiento;
- configurar red;
- configurar audio en segundo plano cuando sea apropiado;
- configurar permisos;
- manejar safe areas;
- manejar orientación;
- probar dispositivos;
- documentar builds;
- documentar publicación.

El Admin no debe incluirse en Capacitor.

FASE 11 — Modo aprender inglés

Objetivo:

Implementar funciones educativas.

Actividades:

- traducción visible u oculta;
- repetición de palabras;
- repetición de oraciones;
- cambio de velocidad;
- vocabulario;
- palabras favoritas;
- historial de palabras;
- significado contextual;
- ejemplos;
- seguimiento de progreso;
- pruebas.

No se debe depender inicialmente de inteligencia artificial para funciones esenciales.

FASE 12 — Calidad, seguridad y rendimiento

Objetivo:

Preparar el proyecto para un uso estable.

Actividades:

- auditoría de seguridad;
 - auditoría de accesibilidad;
 - optimización;
 - lazy loading;
 - compresión;
 - caching;
 - pruebas de carga;
 - pruebas de regresión;
 - manejo de errores global;
 - observabilidad;
 - logging estructurado;
 - métricas;
 - revisión de dependencias;
 - análisis de vulnerabilidades;
 - documentación.
-

FASE 13 — CI/CD y despliegue

Objetivo:

Automatizar validaciones y despliegues.

Actividades:

- GitHub Actions;
- lint;
- type-check;
- tests;
- build;
- Docker;
- despliegue web;
- despliegue API;
- migraciones;
- versionado;
- release notes;
- rollback;
- secretos;
- entornos;
- documentación.

Entornos:

- local;
- development;
- staging;
- production.

FASE 14 — Documentación final y preparación para GitHub

Objetivo:

Hacer que el proyecto pueda ser entendido, instalado, evaluado y continuado por terceros.

Actividades:

- revisar toda la documentación;
- completar README;
- completar guías;
- completar diagramas;
- completar troubleshooting;
- documentar arquitectura;
- documentar API;
- documentar despliegue;
- documentar pruebas;
- documentar contribución;

- preparar capturas;
 - preparar demostración;
 - preparar portafolio.
-

18. DOCUMENTACIÓN EXAGERADAMENTE DETALLADA

La documentación del código fuente debe ser intencionalmente abundante.

Debe estar escrita para un estudiante de desarrollo de software o un desarrollador junior.

No asumas que quien lee conoce la arquitectura, las librerías o las decisiones tomadas.

Cada archivo importante debe explicar:

- para qué existe;
- qué responsabilidad tiene;
- qué no debe hacer;
- quién lo utiliza;
- qué depende de él;
- qué archivos están relacionados;
- qué puede romperse si se modifica incorrectamente.

Cada clase debe documentar:

- propósito;
- responsabilidad;
- dependencias;
- entradas;
- salidas;
- errores;
- ejemplos de uso.

Cada función debe documentar:

- qué hace;
- por qué existe;
- parámetros;
- tipos;
- valor retornado;
- posibles errores;
- efectos secundarios;
- ejemplo;
- notas para modificación.

Cada variable o constante que no sea evidente debe explicar:

- qué representa;

- por qué se necesita;
- unidad de medida;
- valores válidos;
- comportamiento esperado.

Ejemplo de documentación TypeScript:

```
/**
 * Finds the word that should be highlighted for the current audio position.
 *
 * Why this function exists:
 * The audio player only provides the current playback time. The user
interface
 * needs to convert that time into a specific word so it can highlight it and
 * move the reading pointer underneath it.
 *
 * @param currentTimeMs
 * Current audio position expressed in milliseconds.
 *
 * @param speechMarks
 * Ordered list of word timing information returned by Amazon Polly.
 *
 * @returns
 * The active speech mark when a matching word exists, otherwise undefined.
 *
 * @throws
 * This function does not throw errors. Invalid or empty input returns
undefined.
 *
 * Junior developer note:
 * Keep speechMarks sorted by startTimeMs. If the order is changed, the
result
 * may be incorrect.
 */
```

Evita comentarios inútiles como:

```
// Increment counter
counter++;
```

Los comentarios deben explicar intención, arquitectura y decisiones.

19. DOCUMENTACIÓN PARA DESARROLLADORES JUNIOR

Crea una guía central:

docs/development/JUNIOR_DEVELOPER_GUIDE.md

Debe incluir:

- explicación general;
- mapa del repositorio;
- cómo ejecutar cada aplicación;
- cómo depurar;
- cómo agregar una pantalla;
- cómo agregar un endpoint;
- cómo agregar una entidad;
- cómo crear una migración;
- cómo agregar una prueba;
- cómo agregar una traducción;
- cómo agregar una nueva voz;
- cómo modificar el Reader Engine;
- cómo corregir la sincronización;
- cómo investigar un error;
- cómo leer logs;
- cómo revertir cambios;
- cómo crear una rama;
- cómo crear un pull request;
- cómo actualizar documentación.

También crea:

docs/development/WHERE_TO_MAKE_CHANGES.md

Debe funcionar como mapa de mantenimiento.

Ejemplo:

Problema: La palabra resaltada no coincide con el audio.

Revisar:

1. packages/reader-engine/
2. SpeechMarksParser
3. ActiveWordResolver
4. Audio synchronization tests

Problema: La mano aparece desalineada.

Revisar:

1. ReadingPointer component
2. useActiveWordPosition hook
3. CSS transform
4. Auto-scroll behavior

Problema: El audio no se genera.

Revisar:

1. PollyService
2. AWS variables
3. ProcessingJob logs
4. S3 permissions

20. DOCUMENTACIÓN DE TROUBLESHOOTING

Crea documentación específica para problemas.

```
docs/troubleshooting/README.md
docs/troubleshooting/AUDIO.md
docs/troubleshooting/SPEECH_MARKS.md
docs/troubleshooting/AWS.md
docs/troubleshooting/DATABASE.md
docs/troubleshooting/AUTHENTICATION.md
docs/troubleshooting/READER_SYNC.md
docs/troubleshooting/CAPACITOR_ANDROID.md
docs/troubleshooting/CAPACITOR_IOS.md
docs/troubleshooting/OFFLINE_MODE.md
docs/troubleshooting/DEPLOYMENT.md
```

Cada documento debe incluir:

- síntoma;
- causa probable;
- cómo confirmar la causa;
- archivos relacionados;
- comandos de diagnóstico;
- solución;
- validación posterior;
- prevención.

21. DOCUMENTACIÓN DE REQUERIMIENTOS

Crea:

```
docs/requirements/FUNCTIONAL_REQUIREMENTS.md
docs/requirements/NON_FUNCTIONAL_REQUIREMENTS.md
docs/requirements/BUSINESS_RULES.md
docs/requirements/USER_STORIES.md
docs/requirements/USE_CASES.md
docs/requirements/ACCEPTANCE_CRITERIA.md
docs/requirements/TRACEABILITY_MATRIX.md
```

Cada requerimiento debe tener ID.

Ejemplo:

```
FR-READER-001
FR-ADMIN-001
FR-AUDIO-001
FR-OFFLINE-001
NFR-SECURITY-001
NFR-PERFORMANCE-001
NFR-ACCESSIBILITY-001
```

La matriz de trazabilidad debe relacionar:

```
Requirement
User Story
Use Case
Implementation
Test
Status
```

22. REQUERIMIENTOS NO FUNCIONALES

Incluye como mínimo:

Accesibilidad

- WCAG 2.2 AA cuando sea razonable;
- navegación por teclado;
- lectores de pantalla;
- contraste;
- etiquetas;
- tamaños táctiles;
- reducción de movimiento;
- subtítulos textuales;
- mano opcional.

Rendimiento

- carga inicial optimizada;
- lazy loading;
- audio bajo demanda;
- imágenes optimizadas;
- caching;
- respuesta rápida del Reader;
- desplazamiento fluido.

Seguridad

- credenciales protegidas;
- validación;
- autorización;
- protección contra entradas maliciosas;
- políticas de CORS;
- protección de sesiones;
- auditoría;
- secretos fuera del repositorio.

Mantenibilidad

- separación de responsabilidades;
- bajo acoplamiento;
- alta cohesión;
- documentación;
- pruebas;
- tipado estricto;
- arquitectura modular.

Compatibilidad

- navegadores modernos;
- Android;
- iOS;
- diferentes resoluciones;
- orientación vertical y horizontal.

Disponibilidad

El Reader debe seguir funcionando con contenido descargado cuando no exista conexión.

23. EXPERIENCIA DE USUARIO

La experiencia debe ser:

- clara;
- amigable;
- predecible;
- accesible;
- moderna;
- agradable;
- sin sobrecargar al usuario;
- adaptada al público.

Principios:

1. Una acción principal por pantalla.
 2. Los controles importantes deben ser visibles.
 3. El usuario no debe perder su progreso.
 4. Los errores deben explicar qué ocurrió y qué hacer.
 5. El niño no debe acceder accidentalmente a administración.
 6. Las acciones peligrosas deben pedir confirmación.
 7. La reproducción debe continuar de forma estable.
 8. El usuario debe poder ocultar la mano.
 9. El usuario debe poder controlar velocidad y tamaño.
 10. La interfaz debe comunicar estados de descarga y sincronización.
-

24. PRUEBAS

Debes crear:

- pruebas unitarias;
- pruebas de integración;
- pruebas de componentes;
- pruebas de API;
- pruebas E2E;
- pruebas de accesibilidad;
- pruebas de sincronización;
- pruebas de modo offline.

Casos críticos:

- palabra correcta según tiempo;
- salto entre palabras;
- salto entre líneas;
- pausa;
- reanudación;
- cambio de velocidad;
- repetición;
- recuperación de progreso;
- audio faltante;
- Speech Marks inválidos;
- pérdida de conexión;
- descarga interrumpida;
- versión corrupta;
- login inválido;
- permisos insuficientes;
- publicación inválida.

No marques una tarea como terminada sin pruebas apropiadas.

25. ESTÁNDARES DE CÓDIGO

TypeScript

- strict mode;
- evitar `any`;
- interfaces claras;
- tipos compartidos;
- funciones pequeñas;
- nombres descriptivos;
- separación entre UI y lógica;
- manejo explícito de errores.

Python

- type hints;
- funciones pequeñas;
- clases con responsabilidad clara;
- Pydantic para validación;
- separación entre router, service y repository;
- manejo explícito de excepciones;
- documentación abundante.

General

- aplicar SOLID;
 - evitar duplicación;
 - evitar archivos gigantes;
 - evitar funciones con múltiples responsabilidades;
 - evitar valores mágicos;
 - usar constantes;
 - mantener dependencias controladas;
 - documentar decisiones.
-

26. GITHUB

Crea un README profesional que incluya:

- nombre;
- descripción;
- problema;
- solución;
- funcionalidades;
- capturas;
- arquitectura;
- tecnologías;
- estructura;

- requisitos;
- instalación;
- configuración;
- ejecución;
- pruebas;
- Docker;
- Amazon Polly;
- variables de entorno;
- roadmap;
- contribución;
- seguridad;
- licencia;
- estado del proyecto.

Crea también:

```
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SECURITY.md
CHANGELOG.md
LICENSE
```

Incluye plantillas para:

- bugs;
- nuevas funcionalidades;
- mejoras;
- preguntas;
- pull requests.

27. VARIABLES DE ENTORNO

Crea `.env.example`.

Nunca incluyas valores secretos.

Ejemplo:

```
DATABASE_URL=
JWT_SECRET_KEY=
AWS_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_POLLY_DEFAULT_ENGLISH_VOICE=
AWS_POLLY_DEFAULT_SPANISH_VOICE=
```

```
AWS_S3_BUCKET_NAME=  
CORS_ALLOWED_ORIGINS=
```

Explica cada variable en la documentación.

28. REGLAS DE IMPLEMENTACIÓN

1. No construyas todo de una vez.
2. Trabaja por fases.
3. Trabaja por tareas.
4. Revisa dependencias.
5. Mantén estados.
6. Documenta antes y después.
7. Escribe pruebas.
8. No ocultes errores.
9. No marques como completado lo incompleto.
10. No elimines documentación existente sin justificación.
11. No cambies arquitectura sin registrar una decisión.
12. No agregues dependencias sin justificar.
13. No expongas secretos.
14. No uses código temporal sin marcarlo.
15. No dejes TODO sin registrarlo como tarea.
16. No mezcles Admin con Reader.
17. No acoples el Reader directamente con Amazon Polly.
18. No llames Amazon Polly desde el frontend.
19. No dependas de conexión para contenido descargado.
20. No continúes a una fase si faltan entregables críticos.

29. FORMATO DE RESPUESTA DE CODEX EN CADA ITERACIÓN

Al comenzar una iteración responde con:

```
CURRENT PROJECT STATE  
Active phase:  
Active task:  
Task status:  
Dependencies:  
Files to inspect:  
Files to modify:  
Expected result:  
Risks:
```

Durante la implementación debes informar:

IMPLEMENTATION PROGRESS

Completed:
In progress:
Blocked:
Problems found:
Decisions made:

Al finalizar responde con:

SESSION SUMMARY

Phase:
Tasks completed:
Tasks still pending:
Files created:
Files modified:
Tests executed:
Test results:
Documentation updated:
Known issues:
Exact next task:
Recommended next command:

30. PRIMERA EJECUCIÓN

En la primera ejecución no debes empezar inmediatamente a desarrollar componentes visuales o integrar Amazon Polly.

Primero debes:

1. analizar este prompt;
2. crear la estructura documental;
3. crear las fases;
4. crear todas las tareas de la Fase 0;
5. crear el estado inicial;
6. crear el registro de decisiones;
7. crear el registro de riesgos;
8. crear los requerimientos iniciales;
9. crear la arquitectura propuesta;
10. proponer el plan de implementación;
11. indicar cualquier riesgo o inconsistencia;
12. dejar la primera tarea lista para continuar.

Después debes ejecutar la Fase 0 de manera ordenada.

No avances a la Fase 1 hasta que la Fase 0 cumpla todos sus criterios de aceptación.

31. OBJETIVO FINAL

FollowRead debe convertirse en un proyecto profesional de portafolio que demuestre:

- diseño de producto;
- UX/UI;
- React;
- TypeScript;
- Python;
- FastAPI;
- PostgreSQL;
- Amazon Polly;
- Amazon S3;
- sincronización de audio y texto;
- aplicaciones móviles con Capacitor;
- modo offline;
- accesibilidad;
- pruebas;
- seguridad;
- arquitectura;
- documentación;
- CI/CD;
- administración de proyectos por fases.

El código debe ser comprensible para un estudiante, mantenible por un desarrollador junior y suficientemente organizado para ser evaluado por un desarrollador senior o un arquitecto de software.

Comienza ahora con la revisión del prompt, la creación del sistema de seguimiento y la ejecución controlada de la Fase 0.